

JavaScript

▼ Introduction au JS :

Le but du JS est d'enrichir les fonctionnalités d'une page web.

Souvent utilisé pour surveiller les événements et manipuler les éléments d'une page web. (DOM)

Auparavant son utilisation était uniquement destinée au web, dorénavant on l'utilise côté serveur (node.js), les jeux vidéo, applications mobiles, etc.

▼ Historique

Au début, son nom était 'LiveScript' pour des fonctionnalités côté serveur. (Brendan Eich, chez Netscape)

Décision marketing qui fait changer son nom en 'JS'.



Nombreuse confusion avec le langage JAVA, aucune ressemblance !

JavaScript est implémenté dans le navigateur Netscape en 1996.
Microsoft répond avec son propre langage 'JScript'.

En 1997, Netscape soumet JavaScript à l'ECMA International. Premier standard regroupant le JS, le JScript et le ActionScript.

En 1999, mise à jour importante ECMA, évolution des standards en difficulté. (ECMAScript Édition 3)

Cinquième édition en décembre 2009.

L'édition 'ES6' en juin 2015, apporte d'importante nouveauté. Depuis chaque année en juin une nouvelle version est publiée.

Ce qui fait de JS un langage à succès, est sa panoplie de bibliothèque, ainsi que sa normalisation et sa compatibilité sur de nombreux navigateur.

▼ TypeScript

En 2012, première version, langage libre et open source et développé par Microsoft. (Cocréé par Anders Hejlsberg)

Typage statique (facultatif) et les classes. Ainsi que transcompilation (génère du JS natif).

▼ AJAX

Signifie 'Asynchronous JavaScript and XML', permet la communication en arrière plan entre le navigateur et le serveur, afin de permettre à un script de charger des données, sans recharger la page complètement.

Le but étant d'éviter des rechargements de page trop nombreux.



▼ Base du JS :

▼ Intégration :

Il est possible d'intégrer du JS de 2 manières différentes dans une page web.

▼ Balise script

```
<script>
  console.log('hello world');
</script>
```

▼ Script externe

Un peu à la façon d'un fichier CSS, on va indiquer le lien d'un autre fichier, qui sera un fichier '.js' et donc 'JavaScript'.

```
<script src='monfichier.js'></script>
```

▼ Commentaires :

```
//Commentaires une ligne
```

```
/*
Commentaires sur plusieurs
lignes !
*/
```



La balise `<noscript>` permet d'afficher un contenu alternatif lorsque JavaScript est désactivé ou non pris en charge par le navigateur. Elle est souvent utilisée pour informer l'utilisateur que certaines fonctionnalités de la page web ne sont pas disponibles sans JavaScript.

▼ Séparateur d'instructions

En javascript il existe deux séparateurs d'instruction, le point-virgule et le retour à la ligne.

▼ Bloc d'instructions

```
{
  //bloc 1
  console.log('hello word');
}
```

```
{  
  //bloc 2  
  console.log('Bonjour');  
}
```

▼ Débuguer et utiliser la console

Les navigateurs sont équipés d'une console qui permet le débogage.

▼ Méthode de l'objet console

```
console.log(x); //compteur par chaîne$  
console.dir(objet); //liste les propriétés d'un objet  
console.error('Erreur'); //message d'erreur  
console.log(x); //texte ou contenu de variable affiché  
console.table(tableau); //contenu d'un tableau  
console.time('chrono'); //debut chronomètre  
console.timeEnd('chrono'); //fin de chronomètre  
console.warn('Avertissement'); //message d'avertissement
```

▼ Code robuste et actuel

A éviter :

- Eviter les vieux attributs pour l'élément '<script>' (type = '' / language = '').
- Coder des commentaires HTML dans les scripts
- Utiliser la fonction eval()
- Utiliser with()
- Déclarer des variables de manière implicite
- Utiliser document.write()
- Les séparateur de type retour à la ligne

▼ Variables :

Les variables peuvent contenir des objets ou des données de types primitifs (voir plus tard).

▼ Déclaration

Instruction qui provoque la réservation d'un espace mémoire et une attribution de son nom de variable.

```
var prenom = 'Livio'; //Ancienne syntaxe
```

```
let nom = 'Cardinal'; //Déclaration actuel
```

```
const sexe = 'Homme'; //Valeur qui ne peut changer
```

```
let a, b, c; //Déclaration de plusieurs variables sans valeur
```



On préfère utiliser 'let' a 'var', car sa portée est + faible.

```
let x; //Déclaration (undefined)
```

```
x = 4; //Initialisation
```

```
x = 6; //Affectation (déjà initialisé)
```

```
let y = 8; //Déclaration + initialisation
```

```
a = 18; //Pas de déclaration = déclaration implicite
```



Les déclarations implicites sont fortement déconseillé.



L'utilisation d'une variable qui n'est pas déclaré provoque une erreur 'Uncaught ReferenceError'.

▼ Portée de variable

Aussi appelé 'scope'.

Il s'agit des parties du code où la variable est accessible et utilisable.

La portée peut être globale (accessible partout), ou bien locale (accès limité).



La suppression d'une variable locale est détruite à la fin de sa portée, cela provoque une libération de son espace mémoire ainsi que son nom.

Les variables globales, elles, sont supprimées à la fermeture de la page.



Les variables globales, peuvent causer des erreurs, et sont énergivores, il est donc conseillé d'éviter leur utilisation lorsque c'est possible.



'let' permet de déclarer des variables locales dans des blocs de code. En dehors de ceux-ci, les variables seront globales.



'var' déclare des variables locales uniquement dans des fonctions. En dehors il s'agira de variable globale.



Une variable implicite sera toujours globale.

▼ Nom

Règles à respecter :

- Commencer par une lettre, un '_' ou '\$'.
- Composer uniquement de lettres (accent ou non), chiffres, '_' et '\$'.

- Caractères avec accent, ne peuvent pas être suivi d'un chiffre
- Sensible à la case
- Attention au nom réservé :

abstract	boolean	break	byte	case
catch	char	class	const	continue
default	do	double	else	extends
false	final	finally	float	for
function	goto	if	implements	import
in	instanceof	int	interface	long
native	new	null	package	private
protected	public	return	short	static
super	switch	synchronized	this	throw
throws	transient	true	try	typeof
var	void	while	with	



On parle généralement de :

- camelCase ⇒ nomVariable
- Underscore ⇒ nom_variable

▼ Type primitif

4 types primitif de données :

- Nombres (entiers ou avec virgule)
- Chaîne de caractères
- Booléens
- Indéfinis

JavaScript est un langage à typage dynamique, le type des variables peut changer en cours d'exécution.

Typage dynamique ⇒ signifie que le langage détermine automatiquement le type d'une variable en fonction de la valeur qui lui est assignée.

Vérifier le type de la variable avec l'opérateur 'typeof'.

```
console.log(typeof variable);
```

Certaines erreurs sont liés directement au type de la variable.

- NaN ⇒ signifie 'Not a Number'

▼ Conversion de type

```
let x = '4'; //Chaine de caractère puisque guillemet
let y = 'F'; //Chaine de caractère
let z = 5.4; //Reel ⇒ Float

parseInt(x); //Devient un nombre entier (4 ⇒int)
parseInt(y,16); //Devient '15' car F = 15 en base 16
parseFloat(x); //Devient un nombre réel (4.0)
String(z); //Devient une chaine de caractère ('5')

z.toString(); //Devient une chaine de caractère ('5')
z.toFixed(0); //Arrondi le nombre a 0 décimal et le transforme en chaine
x*1 //Devient un nombre
z+' ' //Devient un string
```



Pour les deux dernières lignes il s'agit de conversion un peu plus subtile que l'utilisation de méthodes directement.

▼ Utilisation apostrophe et guillemet

On les utilise pour délimiter les chaînes de caractères.



Comme dans tout les langages de programmation, lorsque l'on utilise les apostrophes ou les guillemets.

Il est possible de rencontrer des erreurs lorsque l'on souhaite introduire des guillemets/apostrophes.

On utilise donc le '\' (caractère d'échappement).

```
let x = 22;
```

```
let xTexte = "J'ai " + x + " ans.";
```

```
let xTexte2 = "J'ai ${x} ans."
```

```
/*
```

Les deux variables sont identiques, dans la première l'utilisation de la variable est faite hors chaîne de caractères avec la concaténation.

La deuxième est une syntaxe qui s'appelle le 'gabarits'.

```
*/
```

▼ Méthodes liées aux strings

```
let x = 'chaîne'
```

```
x.charAt(0); //Donne le caractère à index 0 de x
```

```
x.endsWith('aine'); //Test si x se finit par 'aine'
```

```
x.indexOf('i'); //Renvoie la position du premier 'i' de x
```

```
x.lastIndexOf('i'); //Position du dernier 'i' de x
```

```
x.length //Renvoie le nombre de caractère de x
```

```
x.split('-'); //Crée un tableau en séparant la chaîne à chaque '-'
```

```
x.substring(4); //Renvoie les caractères du 5ème à la fin
```

```
x.substring(4,10); //Du 5ème au 11ème
```

```
x.substr(4,10); //Du 5ème au 15ème
```

```
x.toLowerCase(); //Tout en minuscule
```

```
x.toUpperCase(); //Tout en majuscule  
x.replace('n', 's'); //Remplace le premier n par s
```



Appeler les fonctions ne suffit pas, il est important de stocker les modifications dans de nouvelles variables.

▼ Tableaux indicés

Ici, les indices sont des nombres entiers.

```
//Initialisation  
let tab1 = new Array('Livio', 'Cardinal');  
  
let tab2 = ['Livio', 'Cardianl']; //Même chose  
  
//Affectation  
tab2[2] = '22ans';
```

▼ Tableaux associatif

Ici, les index sont des chaînes de caractère.

```
//Déclaration  
let livio = {'nom' : 'cardinal', 'age' : 22};  
  
//Affectation  
livio['profession'] = 'Etudiant';
```

▼ Parcourir un tableau

- Indiché

```
for (let i = 0; i < tableau.length; i++){  
  console.log(tab[i]);  
}
```

```
}; //Affiche les valeurs du tableau
```



Il est possible d'utiliser un autre type de boucle 'foreach'.

```
tableau.forEach(x ⇒ console.log(x));  
//Affiche tout les éléments du tableau
```

- Associatif

```
for (let i in tableau){  
    console.log(tableau[i]);  
}; //Affiche uniquement les valeurs  
  
for (let i in tableau){  
    console.log('Clé = ${i} : valeur = ${tableau[i]}');  
};  
/*  
Affiche les clés et valeurs sous une certaines mise  
en forme. Avec 'gabarits'.  
*/
```

▼ Tableaux indicés à plusieurs dimensions

Tableau contenant d'autres tableaux.

```
let tab1 = new Array(); //Déclaration du premier tableau  
  
tab1[0] = new Array(); //Premier élément de tab1 = tableau
```

▼ Méthodes et propriétés sur les tableaux

```
tableau.concat(tableau2); //Concaténation de 2 tableaux  
tableau.include(x); //Test si x est dans le tableau
```

```

tableau.indexOf(x); //Renvoie la 1er occurrence et -1 sinon
tableau.join(' '); //Regroupe les éléments en une chaîne de caractère a
tableau.length //Renvoie le nombre d'élément
tableau.pop(); //Supprime le dernier élément
tableau.push(x); //Ajoute x en fin de tableau
tableau.reverse(); //Inverse l'ordre des éléments
tableau.shift(); //Supprime le premier élément du tableau
tableau.sort(); //Trie les éléments par ordre croissant
tableau.splice(ind, x); //Supprime x éléments depuis ind
tableau.unshift(x); //Ajoute x au début du tableau

3 in tableau; //Vérifie si l'indice 3 est présent
length in tableau //Vérifie que la propriété length est présente

tableau.map(fonction); //Crée un nouveau tableau en appliquant une fon

```

▼ Opérateur :

Comme dans tout les langages de programmation.

Ce sont les même opérateurs.

- Arithmétique :

"+" , "-" , etc.

- Incrémentation :

"++i" , "i--" , "i++" et "i--"

- Affection élargie

"+=" , "*=" , etc.

- Concaténation

- Opérateur logiques :

'&&' , '||' , etc.

- Opérateurs relationnels / comparaison :

'>', '>=', etc.

- Opérateurs binaires :

- &
- |
- ^
- ~
- >>
- <<
- >>>

- Opérateur ternaire :

```
let x = 18;  
  
(age >= 18) ? statut = 'Majeur' : statut = 'Mineur';  
  
//Même résultat :  
let statut = (age >= 18) ? 'Majeur' : 'Mineur';
```



Affecte une valeur à une variable en fonction d'une condition, si vrai alors on attribue la valeur à gauche, sinon celle de droite.

▼ Structure conditionnelles :

```
if (condition) {  
    //instruction si condition vrai  
}
```

```
if (condition) {  
    //instruction si condition vrai  
} else {  
    //instruction sinon  
}
```

```
if (condition) {  
    //instruction si condition vrai  
} else if (condition) {  
    //instruction sinon si condition vrai  
} else {  
    //instruction sinon  
}
```

```
switch (variable) {  
    case 1:  
        //instruction si variable = 1  
        break;  
  
    default:  
        //Instruction par défaut  
}
```



Dans le cas d'un 'switch', le mot break indique qu'il faut sortir de la structure conditionnelle.



Souvent lorsqu'il y a plus de 3 conditions, on conseille d'utiliser un switch, pour des raisons de performances.

▼ Structure itératives :

```
while (condition) {  
    //Instruction tant que la condition est vrai  
}  
  
do {  
    //Instruction à exécuter avant d'entrer dans la boucle  
} while (condition); //Vérifie la condition puis rentre dans le do tant que cor  
  
for (affectation; condition; progression) {  
    //Instructions  
}  
  
for (let val in tableau){  
    console.log(tableau[val]);  
    //Affiche les valeurs de mon tableau  
}  
  
for (let val of tableau){  
    console.log(val);  
    //Same shit  
}
```



Il est possible d'utiliser les instructions 'break' et 'continue' dans les boucles.

- **break** ⇒ **Interrompt** immédiatement la boucle et en sort.
- **continue** ⇒ **Passe** à l'itération suivante en **sautant le reste du code** dans la boucle.

▼ Fonctions :

Les fonctions sont des blocs de code réutilisables.

Les fonctions visent à éviter la redondance dans le code et à libérer la mémoire en détruisant les variables locales. (Optimisation des performances)

```
function maFonction (param1, param2){  
  //Instructions de la fonction  
}
```



Les paramètres sont facultatif mais les parenthèses non.



Le nom de la fonction est facultatif, s'il est omis, on appelle cela une fonction anonyme.



Il faut faire appel à la fonction afin d'entraîner son exécution.



On peut utiliser une fonction avant son appel, car JS subit une prélecture par le navigateur.

Il prend donc compte des déclarations de fonction.



Maximum 255 paramètres.



Depuis ES6, il est possible de donner des valeurs par défaut aux paramètres.

▼ Expressions de fonction :

```
let fonction = function () {}
```

Il s'agit d'affecter une fonction à une variable. On appellera la fonction via son nom de variable.

Ces fonctions ne peuvent pas être appelées avant leur déclaration.

▼ Fonction anonyme :

```
function () {  
  //code  
}
```

Usage unique, car sans nom impossible à appeler.

3 façon de les utiliser :

- Expression de fonction
- Auto invocation
- Écoute d'événement

▼ Fonction fléchée :

Depuis ES6, nouvelle signature pour des fonctions anonymes.

```
x ⇒ x + 3; //Avec 1 paramètre  
  
(x, y) ⇒ x + y //Avec 2 paramètres
```

▼ Closure :

Une fonction qui se souvient des variables définies dans sa portée, même après la fin de sa fonction parente.

```
function createCounter() {
  let count = 0; // Variable définie dans la portée de createCounter()

  return function () { // Fonction interne (closure)
    count++; // La fonction interne accède à count
    console.log(count);
  };
}

const increment = createCounter(); // createCounter() s'exécute et retourne une fonction
increment(); // 1
increment(); // 2
increment(); // 3
```

▼ Asynchrone :

JS ne possède pas d'instruction permettant d'interrompre le code, cependant il est possible d'appeler une fonction après un certain laps de temps.

```
setTimeout(fonction, 1000); //Appel de la fonction dans 1 sec

setInterval(fonction, 1000); //Appel de la fonction toutes les secondes
```

Le résultat de la fonction asynchrone est appelé une promesse.



Important de différencier le résultat à une référence de fonction.

```
let x = fonction(); //Résultat de la fonction

let y = fonction; //Référence
```

Dans le cas d'une référence, l'adresse de la fonction sera donné à la variable.

▼ Objet :

JS est un langage orienté objet.

Un objet = ensemble de propriété regroupés.

Si la valeur de la propriété est une fonction alors on parle de méthode.

▼ Objet Window (fenêtre) :

Dit objet racine, il est possible d'utiliser ses méthodes sans préciser son nom.

```
window.alert('oui'); //Message d'alerte
alert('oui'); //Même chose

window.confirm(); //Message avec deux boutons 'ok' 'annuler'
window.prompt(); //Message avec espace ou on peut écrire
```

▼ Objet Math :

Permet d'accéder aux fonctions mathématiques.

```
Math.abs(); //Valeur absolue
Math.sign(); //-1 si -, 1 si + et 0 si nul
Math.ceil(); //Arrondi supérieur
Math.floor(); //Arrondi inférieur
Math.trunc(); //Supprime la partie décimale
Math.round(); //Arrondi au + proche
Math.pow(x, n); //X puissance n
Math.sqrt(); //Racine carrée
Math.cbrt(); //Racine cubique
Math.hypot(x,y); //Hypothénuse
Math.cos(); //Cosinus
Math.sin(); //Sinus
Math.log10(); //Logarithme en base 10
Math.min(); //Renvoie le minimum
Math.max(); //Renvoie le maximum
Math.random(); //Nombre réel aléatoire
```

▼ Null :

Le mot 'null' représente un objet vide.

Et ses propriétés valent 'undefined'.



On peut utiliser le mot 'this', ce mot permet de faire référence directement à l'objet lui-même.

▼ Créer un objet :

```
let obj1 = {'titre' : 'capucine'}; //Sans constructeur

function Bateau(nom, commandant) {
  this.nom = nom;
  this.commandant = commandant;
} //Avec constructeur

let bateau1 = Bateau('oui', 'Livio'); //Initialise un objet bateau
```

▼ Supprimer un objet :

```
delete objet; //Supprime un objet
delete objet.prop; //Supprime une propriété
```

▼ Référence :

Les objets sont stockés par référence, tandis que les variables de type primitif sont stockés par valeur.

```
//Par valeur
let a = 5;
let b = a;
a += 1;

console.log(a); //Affiche 6
console.log(b); //Affiche 5

/*
Ici uniquement la valeur de 'a' a été copiée dans la
variable b. Les modifications effectuées sur a n'auront pas
```

```
d'impact sur 'b'.
```

```
*/
```

```
//Par référence
```

```
let obj1 = {'titre' : 'oui'}
```

```
let obj2 = obj1; //Copie de la référence de l'objet
```

```
obj1['auteur'] = 'Patrick';
```

```
console.dir(obj2);
```

```
//Affiche 'titre' = 'oui', 'auteur' = 'Patrick'
```

```
/*
```

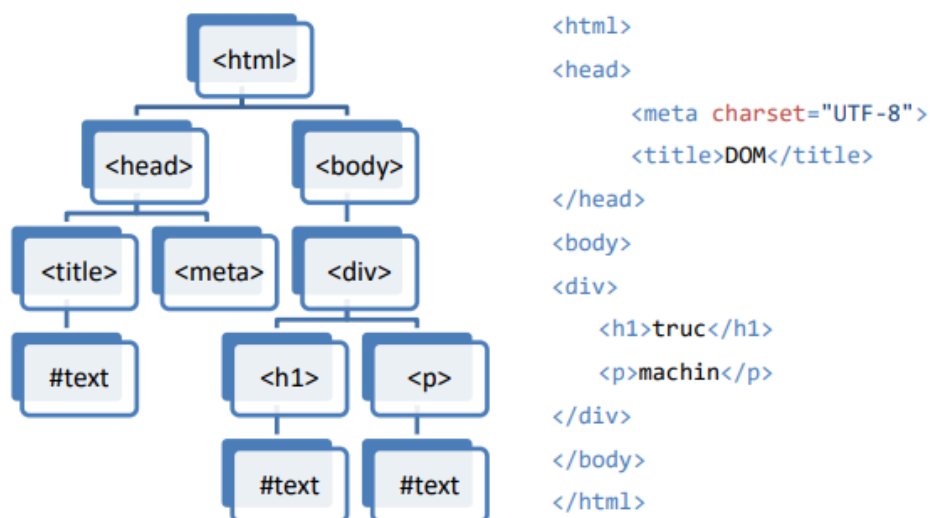
```
Ici toute manipulation sur l'un ou sur l'autre impactera  
les deux.
```

```
*/
```

▼ DOM :

Signifie 'Document Object Model', il s'agit d'une interface de Programmation (API). (Pour XML et HTML)

Détenue par le W3C, actuellement la 3ème version.



Arborescence où les éléments HTML et les textes constituent des nœuds.

▼ Saturation Thread principal :

Processus chargé de la lecture HTML, son CSS, de l'analyse/exécution du JS, etc.

Un DOM surchargé peut provoquer une saturation.

▼ Accès aux éléments :

Toutes les recherches, vont se faire avec l'objet 'document', il est l'objet racine du DOM.

```
//Rechercher un élément
document.getElementById('id'); //Recherche par ID

//Rechercher des éléments
document.getElementsByTagName('p'); //Recherche par balise
document.getElementsByClassName('c'); //Recherche par classe
document.getElementsByName('x'); //Recherche par attribut name

//Par sélecteur CSS
//Pour un élément (le premier trouvé)
document.querySelector('#id');

//Tous les éléments
document.querySelectorAll('.classe');

//Par parentalité
x.children //éléments enfant de x
x.firstChild //premier enfant de x
x.lastElementChild //dernier enfant de x
x.parentElement //parent de x
x.nextElementSibling //prochain frère
x.previousElementSibling //frère précédent

//Accès direct
```

```
document.body //body
document.forms //Tableau contenant tous les formulaires
document.script //Tableau contenant tous les scripts
```



Par défaut, la recherche se fait dans tout le document. Cependant il est possible d'être plus précis dans la recherche en combinant ces propriétés.

```
document.getElementById('x').getElementsByClassName('y');

/*
Ici, on va rechercher un élément qui a comme classe y, et qui
est contenu dans un élément qui a un id 'y'.
*/
```



Il n'est pas conseillé mais possible d'accéder un élément qui possède un id par une variable globales qui portent le même nom.

▼ Modifier le contenu des éléments :

```
x.innerHTML = ''; //Modifie l'ensemble de l'élément y compris les enfants/
x.innerText = ''; //Modifie uniquement le texte
x.textContent = ''; //Modifie uniquement le texte même caché

x.prepend(''); //Ajout de contenu au début de l'élément
x.append(''); //Ajout de contenu en fin d'élément

x.remove(); //Supprime un élément
```



Informations supplémentaires :

<code>.childNodes</code>	nœuds enfants
<code>.firstChild</code>	premier nœud enfant
<code>.lastChild</code>	dernier nœud enfant
<code>.parentNode</code>	nœud parent
<code>.previousSibling</code>	nœud précédent d'un type (nœud de même niveau)
<code>.nextSibling</code>	prochain nœud d'un type (nœud de même niveau)
<code>.nodeName</code>	nom du nœud
<code>.nodeValue</code>	contenu du nœud
<code>.nodeType</code>	type du nœud

▼ Propriété CSS et HTML :

Lorsque l'on accède à des éléments en JS, on les récupère sous forme d'objet.

Comme tout type d'objet, ils possèdent des propriétés, qui seront modifiables.

```
//x étant un élément type image
x.src = ""; //Change la source
x.width = ; //Change la largeur

//y étant un élément de type p, et on modifie le css
y.style.color = 'red'; //Met la couleur en rouge
y.style.font-size = ; //Change la taille
```



On peut donc modifier les éléments HTML avec le JavaScript. Une autre méthode peut-être utilisée à ses fins également, 'getAttribute()'.

```
//x étant un élément de type image
x.setAttribute('src', ''); //Change la source
```



```
//Également possible de récupérer la valeur d'un attribut  
x.getAttribute('src'); //Récupère la source
```



L'attribut 'class' d'un élément peut en contenir plusieurs, s'ils sont séparés par un espace.

Il existe plusieurs manipulations possible concernant cet attribut.

```
x.classList.add('y'); //Ajoute une classe  
x.classList.remove('y'); //Retire une classe  
x.classList.replace('y', 'z'); //Remplace une classe 'y' par 'z'  
x.classList.toggle('y'); //Alterne une classe  
x.classList.contains('y'); //Vérifie si la classe 'y' est présente
```

▼ Événement :

Actions détectables de l'utilisateur.

Évènements JavaScript	Attributs HTML	Descriptions
click	onclick	Le clic
dblclick	ondblclick	Le double-clic
mouseover	onmouseover	Survol
mouseout	onmouseout	Fin du survol
mouseenter	onmouseenter	Survol sans propagation
mouseleave	onmouseleave	Fin du survol sans propagation
mousedown	onmousedown	L'enfoncement du bouton de la souris
mouseup	onmouseup	Le relâchement du bouton de la souris
mousemove	onmousemove	Le mouvement du curseur sur un élément
pointerdown	onpointerdown	Extension des évènements « <i>mouse</i> » et fusion avec les évènements « <i>touch</i> » consacrés aux mobiles mais incompatibles sur Safari, les évènements « <i>pointer</i> » se préoccupent de la souris, du stylo ou stylet et des points de contact sur écran tactile.
pointerup	onpointerup	
pointerover	onpointerover	
pointerout	onpointerout	
pointerenter	onpointerenter	
pointerleave	onpointerleave	
pointermove	onpointermove	
load	onload	Le chargement d'une ressource et de ses ressources dépendantes ; souvent utilisé sur <i>body</i>
unload	onunload	La fermeture de la page
scroll	onscroll	L'utilisation des barres de défilement
resize	onresize	Le redimensionnement de la page
focus	onfocus	L'activation d'un champ du formulaire. L'évènement <i>focusin</i> est semblable, mais avec propagation.
blur	onblur	La désactivation d'un champ de formulaire. L'évènement <i>focusout</i> est semblable, mais avec propagation.
change	onchange	Le changement de la valeur d'un champ
input	oninput	L'entrée d'un caractère dans un champ texte
select	onselect	La sélection d'une <i>option</i> dans une liste de sélection
submit	onsubmit	L'envoi d'un formulaire (uniquement sur <form>)
keydown	onkeydown	Touche du clavier enfoncée
keypress	onkeypress	Touche du clavier pressée (différente pour <i>caps lock</i>)
keyup	onkeyup	Touche du clavier relâchée

▼ Écoute des événements :

```
x.addEventListener('click', function(){
  console.log('click')
});
//Affichera click en console à chaque clique

x.removeEventListener('click', function(){
  console.log('click')
```

```
});  
//Retire l'ecoute de l'événement
```



Il est possible également de réaliser l'écoute des événements directement depuis le fichier HTML.

Voir deuxième colonne du tableau.

```
<img onclick='fonction()' src='' alt=''>
```

```
<!--
```

Lorsque l'on clique sur l'image, on appellera la fonction.

→

▼ Gestionnaire d'événement :

⇒ suite page 39, point D